

- Frontend Implementation Plan for GTD Search App
 - Project Overview
 - Current Implementation Status
 - Implementation Phases
 - Phase 1: API Integration (4-6 weeks)
 - 1. API Client Setup
 - 2. Task API Service
 - 3. Workspace API Service
 - 4. Update SearchView Component
 - 5. Update App Component with API Integration
 - Phase 2: Enhanced UI Components (4-6 weeks)
 - 1. Task Detail Component
 - 2. Enhanced Search Results Component
 - 3. Enhanced Workspace Panel Component
 - Phase 3: Advanced Features (4-6 weeks)
 - 1. Keyboard Shortcuts
 - 2. Workspace Layout Persistence
 - 3. Vector Search Integration
 - Phase 4: Performance Optimizations (2-4 weeks)
 - 1. Virtualized Lists for Search Results
 - 2. Debounced Search
 - 3. Memoized Components
 - Project Structure
 - Implementation Timeline
 - Phase 1: API Integration (4-6 weeks)
 - Phase 2: Enhanced UI Components (4-6 weeks)
 - Phase 3: Advanced Features (4-6 weeks)
 - Phase 4: Performance Optimizations (2-4 weeks)
 - Dependencies
 - Future Enhancements
 - 1. Advanced Search Features
 - 2. Task Management
 - 3. Visualization and Analytics
 - 4. Collaboration Features
 - 5. Integration with External Tools
 - Implementation Notes
 - API Integration Strategy

- [UI/UX Considerations](#)
- [Performance Optimization Techniques](#)

Frontend Implementation Plan for GTD Search App

Project Overview

- **Data Source:** FastAPI backend connecting to PostgreSQL with todo data
- **UI Framework:** React with TypeScript
- **Time Constraint:** 4-8 hours per week development time
- **Current Progress:** Basic workspace view and search view implemented
- **Next Focus:** Connecting to FastAPI backend and enhancing UI

Current Implementation Status

The current codebase includes:

- Basic search interface with filtering capabilities
- Simple workspace view with panels and tabs
- Drag and drop functionality for task management
- Task detail view for displaying task information

Implementation Phases

Phase 1: API Integration (4-6 weeks)

1. API Client Setup

```
// src/api/client.ts
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_BASE_URL ||
'http://localhost:8000/api';
```

```
const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

export default apiClient;
```

2. Task API Service

```
// src/api/taskService.ts
import apiClient from './client';
import { Task, SearchParams } from '../types';

export interface ApiSearchParams {
  query?: string;
  list_name?: string;
  folder_name?: string;
  status?: string;
  due_date_start?: string;
  due_date_end?: string;
  completed?: boolean;
  page: number;
  page_size: number;
}

export interface ApiTask {
  todo_id: string;
  todo_title: string;
  todo_content: string;
  todo_list_name: string;
  todo_folder_name: string;
  todo_status: string;
  todo_duedate: string;
  todo_tags: string;
  created_at: string;
  updated_at: string;
}

// Convert API task format to our app's Task format
const mapApiTaskToTask = (apiTask: ApiTask): Task => {
  return {
    id: apiTask.todo_id,
    title: apiTask.todo_title,
    content: apiTask.todo_content,
    status: apiTask.todo_status === '1' ? 'completed' :
      apiTask.todo_status === '2' ? 'in_progress' : 'pending',
    priority: 'medium', // Default if not provided by API
    dueDate: apiTask.todo_duedate,
    folder: apiTask.todo_folder_name,
    list: apiTask.todo_list_name,
    tags: apiTask.todo_tags ? apiTask.todo_tags.split(',') : [],
    createdAt: apiTask.created_at,
```

```

        updatedAt: apiTask.updated_at
    };
};

// Convert our search params to API format
const mapSearchParamsToApiParams = (params: SearchParams): ApiSearchParams
=> {
    return {
        query: params.query,
        list_name: params.filters.list,
        folder_name: params.filters.folder,
        status: params.filters.status?.length === 1 ? params.filters.status[0] :
undefined,
        due_date_start: params.filters.dateFrom,
        due_date_end: params.filters.dateTo,
        completed: params.filters.status?.includes('completed') || undefined,
        page: 1,
        page_size: 50
    };
};

export const searchTasks = async (searchParams: SearchParams):
Promise<Task[]> => {
    try {
        const apiParams = mapSearchParamsToApiParams(searchParams);
        const response = await apiClient.get('/todos/search', { params:
apiParams });
        return response.data.map(mapApiTaskToTask);
    } catch (error) {
        console.error('Error searching tasks:', error);
        throw error;
    }
};

export const getTaskById = async (taskId: string): Promise<Task> => {
    try {
        const response = await apiClient.get(`/todos/${taskId}`);
        return mapApiTaskToTask(response.data);
    } catch (error) {
        console.error(`Error fetching task ${taskId}:`, error);
        throw error;
    }
};

export const getMetadata = async () => {
    try {
        const [folders, lists, statuses, tags] = await Promise.all([
            apiClient.get('/folders'),
            apiClient.get('/lists'),
            apiClient.get('/statuses'),
            apiClient.get('/tags')
        ]);

        return {
            folders: folders.data,
            lists: lists.data,
            statuses: statuses.data,

```

```

        tags: tags.data
    };
} catch (error) {
    console.error('Error fetching metadata:', error);
    throw error;
}
};

```

3. Workspace API Service

```

// src/api/workspaceService.ts
import apiClient from './client';
import { WorkspaceState } from '../types';
import { v4 as uuidv4 } from 'uuid';

// Default user ID - in a real app, this would come from authentication
const USER_ID = 'default-user';

export const saveWorkspace = async (workspace: WorkspaceState):
Promise<WorkspaceState> => {
    try {
        const workspaceData = {
            id: workspace.id || uuidv4(),
            user_id: USER_ID,
            name: 'Default Workspace',
            open_tasks: workspace.openTasks,
            pane_groups: workspace.paneGroups.map(pg => ({
                id: pg.id,
                task_ids: pg.taskIds,
                active_task_id: pg.activeTaskId,
                size: pg.size
            })),
            active_task_id: workspace.activeTaskId,
            created_at: new Date().toISOString(),
            updated_at: new Date().toISOString()
        };

        const response = await apiClient.post('/workspaces', workspaceData);

        // Map the response back to our app's format
        return {
            id: response.data.id,
            openTasks: response.data.open_tasks,
            paneGroups: response.data.pane_groups.map(pg => ({
                id: pg.id,
                taskIds: pg.task_ids,
                activeTaskId: pg.active_task_id,
                size: pg.size
            })),
            activeTaskId: response.data.active_task_id
        };
    } catch (error) {
        console.error('Error saving workspace:', error);
        throw error;
    }
}

```

```

    }
  };

export const loadWorkspace = async (workspaceId: string):
Promise<WorkspaceState> => {
  try {
    const response = await apiClient.get(`/workspaces/${workspaceId}`);

    // Map the response to our app's format
    return {
      id: response.data.id,
      openTasks: response.data.open_tasks,
      paneGroups: response.data.pane_groups.map(pg => ({
        id: pg.id,
        taskIds: pg.task_ids,
        activeTaskId: pg.active_task_id,
        size: pg.size
      })),
      activeTaskId: response.data.active_task_id
    };
  } catch (error) {
    console.error(`Error loading workspace ${workspaceId}:`, error);
    throw error;
  }
};

export const updateWorkspace = async (workspaceId: string, workspace:
WorkspaceState): Promise<WorkspaceState> => {
  try {
    const workspaceData = {
      id: workspaceId,
      user_id: USER_ID,
      name: 'Default Workspace',
      open_tasks: workspace.openTasks,
      pane_groups: workspace.paneGroups.map(pg => ({
        id: pg.id,
        task_ids: pg.taskIds,
        active_task_id: pg.activeTaskId,
        size: pg.size
      })),
      active_task_id: workspace.activeTaskId,
      updated_at: new Date().toISOString()
    };

    const response = await apiClient.put(`/workspaces/${workspaceId}`,
workspaceData);

    // Map the response back to our app's format
    return {
      id: response.data.id,
      openTasks: response.data.open_tasks,
      paneGroups: response.data.pane_groups.map(pg => ({
        id: pg.id,
        taskIds: pg.task_ids,
        activeTaskId: pg.active_task_id,
        size: pg.size
      })),

```

```

        activeTaskId: response.data.active_task_id
    };
} catch (error) {
    console.error(`Error updating workspace ${workspaceId}:`, error);
    throw error;
}
};

```

4. Update SearchView Component

```

// src/components/SearchView.tsx
import React, { useEffect, useState } from 'react';
import { Task, SearchParams } from '../types';
import SearchBar from './SearchBar';
import FilterPanel from './FilterPanel';
import TaskList from './TaskList';
import { searchTasks, getMetadata } from '../api/taskService';

interface SearchViewProps {
    tasks: Task[];
    searchParams: SearchParams;
    onSearchChange: (params: SearchParams) => void;
    onAddToWorkspace: (task: Task) => void;
    openTaskIds: string[];
    onNavigateToWorkspace: () => void;
}

const SearchView: React.FC<SearchViewProps> = ({
    searchParams,
    onSearchChange,
    onAddToWorkspace,
    openTaskIds,
    onNavigateToWorkspace
}) => {
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState<string | null>(null);
    const [searchResults, setSearchResults] = useState<Task[]>([]);
    const [metadata, setMetadata] = useState({
        folders: [],
        lists: [],
        statuses: [],
        tags: []
    });

    // Load metadata when component mounts
    useEffect(() => {
        const loadMetadata = async () => {
            try {
                const data = await getMetadata();
                setMetadata(data);
            } catch (err) {
                setError('Failed to load metadata. Please try again later.');
```

```

    loadMetadata();
  }, []);

// Perform search when search params change
useEffect(() => {
  const performSearch = async () => {
    setLoading(true);
    setError(null);

    try {
      const results = await searchTasks(searchParams);
      setSearchResults(results);
    } catch (err) {
      setError('Search failed. Please try again later.');
```

setSearchResults([]);

```
    } finally {
      setLoading(false);
    }
  };

  performSearch();
}, [searchParams]);

const handleSearchQueryChange = (query: string) => {
  onSearchChange({
    ...searchParams,
    query
  });
};

const handleFilterChange = (filters: any) => {
  onSearchChange({
    ...searchParams,
    filters
  });
};

const handleTaskClick = (task: Task) => {
  onAddToWorkspace(task);
};

return (
  <div className="search-view">
    <div className="search-panel">
      <SearchBar
        searchParams={searchParams}
        onSearchChange={handleSearchQueryChange}
      />
      <FilterPanel
        searchParams={searchParams}
        onFilterChange={handleFilterChange}
        availableFolders={metadata.folders}
        availableLists={metadata.lists}
        availableStatuses={metadata.statuses}
        availableTags={metadata.tags}
      />
    </div>
  </div>
);

```



```

</div>

<div className="search-results">
  {loading && <div className="loading">Loading...</div>}
  {error && <div className="error">{error}</div>}
  {!loading && !error && (
    <>
      <div className="results-header">
        <h2>Search Results ({searchResults.length})</h2>
        {openTaskIds.length > 0 && (
          <button onClick={onNavigateToWorkspace}>
            Go to Workspace ({openTaskIds.length})
          </button>
        )}
      </div>
      <TaskList
        tasks={searchResults}
        onTaskClick={handleTaskClick}
        openTaskIds={openTaskIds}
      />
    </>
  )}
</div>
</div>
);
};

export default SearchView;

```

5. Update App Component with API Integration

```

// src/App.tsx
import React, { useState, useEffect } from 'react';
import { DndProvider } from 'react-dnd';
import { HTML5Backend } from 'react-dnd-html5-backend';
import { v4 as uuidv4 } from 'uuid';
import SearchView from './components/SearchView';
import WorkspaceView from './components/WorkspaceView';
import { Task, SearchParams, WorkspaceState } from './types';
import { searchTasks, getTaskById } from './api/taskService';
import { saveWorkspace, loadWorkspace, updateWorkspace } from
'./api/workspaceService';

const App: React.FC = () => {
  const [currentView, setCurrentView] = useState<'search' | 'workspace'>
('search');
  const [tasks, setTasks] = useState<Task[]>([]);
  const [searchParams, setSearchParams] = useState<SearchParams>({
    query: '',
    filters: {}
  });

  // Initialize workspace with a single empty pane group
  const [workspace, setWorkspace] = useState<WorkspaceState>({

```

```

    id: uuidv4(),
    openTasks: [],
    paneGroups: [{ id: uuidv4(), taskIds: [], activeTaskId: undefined }]
  });

// Load initial search results
useEffect(() => {
  const loadInitialTasks = async () => {
    try {
      const results = await searchTasks({ query: '', filters: {} });
      setTasks(results);
    } catch (error) {
      console.error('Failed to load initial tasks:', error);
    }
  };

  loadInitialTasks();
}, []);

// Try to load saved workspace
useEffect(() => {
  const loadSavedWorkspace = async () => {
    try {
      // In a real app, you'd get the workspace ID from user preferences
      const savedWorkspace = await loadWorkspace('default-workspace');
      setWorkspace(savedWorkspace);
    } catch (error) {
      console.log('No saved workspace found, using default');
    }
  };

  loadSavedWorkspace();
}, []);

// Add a task to workspace
const addTaskToWorkspace = async (task: Task) => {
  if (workspace.openTasks.includes(task.id)) {
    // Task already open, just navigate to workspace
    setCurrentView('workspace');
    return;
  }

  // Add to first pane group by default
  const updatedPaneGroups = [...workspace.paneGroups];
  updatedPaneGroups[0].taskIds.push(task.id);

  // If no active task in this pane, make this one active
  if (!updatedPaneGroups[0].activeTaskId) {
    updatedPaneGroups[0].activeTaskId = task.id;
  }

  const updatedWorkspace = {
    ...workspace,
    openTasks: [...workspace.openTasks, task.id],
    paneGroups: updatedPaneGroups,
    activeTaskId: task.id
  };
};

```

```

    setWorkspace(updatedWorkspace);

    // Save the updated workspace
    try {
        await updateWorkspace(workspace.id, updatedWorkspace);
    } catch (error) {
        console.error('Failed to save workspace:', error);
    }

    setCurrentView('workspace');
};

// Handle search parameter changes
const handleSearchChange = async (newParams: SearchParams) => {
    setSearchParams(newParams);

    try {
        const results = await searchTasks(newParams);
        setTasks(results);
    } catch (error) {
        console.error('Search failed:', error);
    }
};

// Save workspace state
const saveWorkspaceState = async (updatedWorkspace: WorkspaceState) => {
    setWorkspace(updatedWorkspace);

    try {
        await updateWorkspace(workspace.id, updatedWorkspace);
    } catch (error) {
        console.error('Failed to save workspace:', error);
    }
};

return (
    <DndProvider backend={HTML5Backend}>
        <div className="app">
            <header>
                <h1>GTD Search App</h1>
                <nav>
                    <button
                        className={currentView === 'search' ? 'active' : ''}
                        onClick={() => setCurrentView('search')}
                    >
                        Search
                    </button>
                    <button
                        className={currentView === 'workspace' ? 'active' : ''}
                        onClick={() => setCurrentView('workspace')}
                    >
                        Workspace ({workspace.openTasks.length})
                    </button>
                </nav>
            </header>

```

```

    <main>
      {currentView === 'search' ? (
        <SearchView
          tasks={tasks}
          searchParams={searchParams}
          onSearchChange={handleSearchChange}
          onAddToWorkspace={addTaskToWorkspace}
          openTaskIds={workspace.openTasks}
          onNavigateToWorkspace={() => setCurrentView('workspace')}
        />
      ) : (
        <WorkspaceView
          workspace={workspace}
          onWorkspaceChange={saveWorkspaceState}
          onNavigateToSearch={() => setCurrentView('search')}
          fetchTask={getTaskById}
        />
      )}
    </main>
  </div>
</DndProvider>
);
};

export default App;

```

Phase 2: Enhanced UI Components (4-6 weeks)

1. Task Detail Component

```

// src/components/TaskDetail.tsx
import React, { useState, useEffect } from 'react';
import { Task } from '../types';
import ReactMarkdown from 'react-markdown';
import { format } from 'date-fns';

interface TaskDetailProps {
  taskId: string;
  fetchTask: (id: string) => Promise<Task>;
  onClose: () => void;
}

const TaskDetail: React.FC<TaskDetailProps> = ({ taskId, fetchTask, onClose }) => {
  const [task, setTask] = useState<Task | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const loadTask = async () => {

```

```

    setLoading(true);
    setError(null);

    try {
      const taskData = await fetchTask(taskId);
      setTask(taskData);
    } catch (err) {
      setError('Failed to load task details');
    } finally {
      setLoading(false);
    }
  };

  loadTask();
}, [taskId, fetchTask]);

if (loading) return <div className="task-detail loading">Loading task
details...</div>;
if (error) return <div className="task-detail error">{error}</div>;
if (!task) return <div className="task-detail error">Task not found</div>;

return (
  <div className="task-detail">
    <header>
      <h2>{task.title}</h2>
      <button className="close-button" onClick={onClose}>×</button>
    </header>

    <div className="task-metadata">
      <div className="metadata-item">
        <span className="label">Status:</span>
        <span className={`status ${task.status}`}>{task.status}</span>
      </div>

      {task.dueDate && (
        <div className="metadata-item">
          <span className="label">Due:</span>
          <span className="due-date">{format(new Date(task.dueDate),
'PPP')}</span>
        </div>
      )}

      <div className="metadata-item">
        <span className="label">List:</span>
        <span className="list">{task.list}</span>
      </div>

      <div className="metadata-item">
        <span className="label">Folder:</span>
        <span className="folder">{task.folder}</span>
      </div>
    </div>

    {task.tags && task.tags.length > 0 && (
      <div className="task-tags">
        {task.tags.map(tag => (
          <span key={tag} className="tag">#{tag}</span>

```

```

    ))}
  </div>
)}

<div className="task-content">
  <ReactMarkdown>{task.content || 'No content'}</ReactMarkdown>
</div>

<div className="task-actions">
  <a
    href=
    {`ticktick://ticktick.com/webapp/#p/${task.list}/tasks/${task.id}`}
    className="external-link"
    target="_blank"
    rel="noopener noreferrer"
  >
    Open in TickTick
  </a>
</div>
</div>
);
};

export default TaskDetail;

```

2. Enhanced Search Results Component

```

// src/components/SearchResults.tsx
import React, { useState } from 'react';
import { Task } from '../types';
import { format } from 'date-fns';

interface SearchResultsProps {
  tasks: Task[];
  onTaskClick: (task: Task) => void;
  openTaskIds: string[];
}

type GroupBy = 'none' | 'folder' | 'list' | 'status' | 'dueDate';
type SortBy = 'relevance' | 'title' | 'dueDate' | 'createdAt';

const SearchResults: React.FC<SearchResultsProps> = ({ tasks, onTaskClick,
openTaskIds }) => {
  const [groupBy, setGroupBy] = useState<GroupBy>('none');
  const [sortBy, setSortBy] = useState<SortBy>('relevance');
  const [sortDirection, setSortDirection] = useState<'asc' | 'desc'>
('desc');
  const [viewMode, setViewMode] = useState<'compact' | 'detailed'>
('compact');

  // Sort tasks
  const sortedTasks = [...tasks].sort((a, b) => {
    if (sortBy === 'title') {
      return sortDirection === 'asc'

```

```

        ? a.title.localeCompare(b.title)
        : b.title.localeCompare(a.title);
    } else if (sortBy === 'dueDate') {
        if (!a.dueDate) return sortDirection === 'asc' ? -1 : 1;
        if (!b.dueDate) return sortDirection === 'asc' ? 1 : -1;
        return sortDirection === 'asc'
            ? new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime()
            : new Date(b.dueDate).getTime() - new Date(a.dueDate).getTime();
    } else if (sortBy === 'createdAt') {
        return sortDirection === 'asc'
            ? new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime()
            : new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime();
    }
    // Default to relevance (as returned by API)
    return 0;
});

// Group tasks
const groupedTasks: Record<string, Task[]> = {};

if (groupBy === 'none') {
    groupedTasks['All Tasks'] = sortedTasks;
} else {
    sortedTasks.forEach(task => {
        let groupKey = '';

        if (groupBy === 'folder') {
            groupKey = task.folder || 'No Folder';
        } else if (groupBy === 'list') {
            groupKey = task.list || 'No List';
        } else if (groupBy === 'status') {
            groupKey = task.status || 'No Status';
        } else if (groupBy === 'dueDate') {
            if (!task.dueDate) {
                groupKey = 'No Due Date';
            } else {
                const dueDate = new Date(task.dueDate);
                const today = new Date();
                const tomorrow = new Date(today);
                tomorrow.setDate(tomorrow.getDate() + 1);

                if (dueDate < today) {
                    groupKey = 'Overdue';
                } else if (
                    dueDate.getDate() === today.getDate() &&
                    dueDate.getMonth() === today.getMonth() &&
                    dueDate.getFullYear() === today.getFullYear()
                ) {
                    groupKey = 'Today';
                } else if (
                    dueDate.getDate() === tomorrow.getDate() &&
                    dueDate.getMonth() === tomorrow.getMonth() &&
                    dueDate.getFullYear() === tomorrow.getFullYear()
                ) {
                    groupKey = 'Tomorrow';
                } else {
                    groupKey = format(dueDate, 'MMMM yyyy');
                }
            }
        }
    });
}

```

```

    }
  }
}

if (!groupedTasks[groupKey]) {
  groupedTasks[groupKey] = [];
}

groupedTasks[groupKey].push(task);
});
}

return (
  <div className="search-results">
    <div className="results-controls">
      <div className="view-controls">
        <label>View:</label>
        <button
          className={viewMode === 'compact' ? 'active' : ''}
          onClick={() => setViewMode('compact')}
        >
          Compact
        </button>
        <button
          className={viewMode === 'detailed' ? 'active' : ''}
          onClick={() => setViewMode('detailed')}
        >
          Detailed
        </button>
      </div>

      <div className="group-controls">
        <label>Group by:</label>
        <select
          value={groupBy}
          onChange={(e) => setGroupBy(e.target.value as GroupBy)}
        >
          <option value="none">None</option>
          <option value="folder">Folder</option>
          <option value="list">List</option>
          <option value="status">Status</option>
          <option value="dueDate">Due Date</option>
        </select>
      </div>

      <div className="sort-controls">
        <label>Sort by:</label>
        <select
          value={sortBy}
          onChange={(e) => setSortBy(e.target.value as SortBy)}
        >
          <option value="relevance">Relevance</option>
          <option value="title">Title</option>
          <option value="dueDate">Due Date</option>
          <option value="createdAt">Created Date</option>
        </select>
        <button onClick={() => setSortDirection(sortDirection === 'asc' ?

```



```

'desc' : 'asc'))>
    {sortDirection === 'asc' ? '↑' : '↓'}
  </button>
</div>
</div>

<div className="results-count">
  Found {tasks.length} tasks
</div>

<div className={`results-list ${viewMode}`}>
  {Object.entries(groupedTasks).map(([group, groupTasks]) => (
    <div key={group} className="task-group">
      <h3 className="group-title">{group} ({groupTasks.length})</h3>
      <div className="group-tasks">
        {groupTasks.map(task => (
          <div
            key={task.id}
            className={`task-item ${openTaskIds.includes(task.id) ?
'open' : ''}`}
            onClick={() => onTaskClick(task)}
          >
            <div className="task-title">
              {task.title}
              {openTaskIds.includes(task.id) && (
                <span className="open-indicator">Open</span>
              )}
            </div>

            {viewMode === 'detailed' && (
              <>
                <div className="task-metadata">
                  {task.dueDate && (
                    <span className="due-date">
                      Due: {format(new Date(task.dueDate), 'MMM d')}
                    </span>
                  )}
                <span className={`status ${task.status}`}>
                  {task.status}
                </span>
                <span className="list">
                  {task.list}
                </span>
              </div>

                {task.content && (
                  <div className="task-preview">
                    {task.content.substring(0, 100)}
                    {task.content.length > 100 ? '...' : ''}
                  </div>
                )}

                {task.tags && task.tags.length > 0 && (
                  <div className="task-tags">
                    {task.tags.map(tag => (
                      <span key={tag} className="tag">#{tag}</span>
                    )}}
                )}
              </div>
            )}
          </div>
        )}
      </div>
    )}
  )}

```

```

        </div>
      )}
    </>
  )}
</div>
)}
</div>
</div>
)}
</div>
</div>
);
};

export default SearchResults;

```

3. Enhanced Workspace Panel Component

```

// src/components/WorkspacePanel.tsx
import React, { useState, useEffect } from 'react';
import { useDrop } from 'react-dnd';
import { Task } from '../types';
import TaskTab from '../TaskTab';
import TaskDetail from '../TaskDetail';

interface WorkspacePanelProps {
  id: string;
  taskIds: string[];
  activeTaskId?: string;
  onActivateTask: (taskId: string) => void;
  onCloseTask: (taskId: string) => void;
  onDropTask: (taskId: string) => void;
  fetchTask: (id: string) => Promise<Task>;
}

const WorkspacePanel: React.FC<WorkspacePanelProps> = ({
  id,
  taskIds,
  activeTaskId,
  onActivateTask,
  onCloseTask,
  onDropTask,
  fetchTask
}) => {
  const [tasks, setTasks] = useState<Record<string, Task>>({});
  const [loading, setLoading] = useState<Record<string, boolean>>({});
  const [errors, setErrors] = useState<Record<string, string>>({});

  // Set up drop target for dragged tabs
  const [{ isOver }, drop] = useDrop({
    accept: 'task-tab',
    drop: (item: { taskId: string, sourcePanel: string }) => {
      if (item.sourcePanel !== id) {
        onDropTask(item.taskId);

```

```

    }
  },
  collect: (monitor) => ({
    isOver: !!monitor.isOver()
  })
});

// Load task data for all task IDs
useEffect(() => {
  const loadTasks = async () => {
    const newTasks = { ...tasks };
    const newLoading = { ...loading };
    const newErrors = { ...errors };

    for (const taskId of taskIds) {
      if (!tasks[taskId] && !loading[taskId]) {
        newLoading[taskId] = true;
        setLoading(newLoading);

        try {
          const taskData = await fetchTask(taskId);
          newTasks[taskId] = taskData;
          newLoading[taskId] = false;
        } catch (error) {
          newErrors[taskId] = 'Failed to load task';
          newLoading[taskId] = false;
        }
      }
    }

    setTasks(newTasks);
    setLoading(newLoading);
    setErrors(newErrors);
  };

  loadTasks();
}, [taskIds, tasks, loading, errors, fetchTask]);

return (
  <div
    className={`workspace-panel ${isOver ? 'drop-target' : ''}`}
    ref={drop}
  >
    <div className="panel-tabs">
      {taskIds.map(taskId => (
        <TaskTab
          key={taskId}
          taskId={taskId}
          title={tasks[taskId]?.title || 'Loading...'}
          isActive={taskId === activeTaskId}
          isLoading={loading[taskId]}
          hasError={!!errors[taskId]}
          onActivate={() => onActivateTask(taskId)}
          onClose={() => onCloseTask(taskId)}
          panelId={id}
        />
      ))}
    </div>
  </div>
)

```

```

</div>

<div className="panel-content">
  {taskIds.length === 0 ? (
    <div className="empty-panel">
      <p>No tasks open in this panel</p>
      <p>Drag tasks here from search results or other panels</p>
    </div>
  ) : activeTaskId ? (
    <TaskDetail
      taskId={activeTaskId}
      fetchTask={fetchTask}
      onClose={() => onCloseTask(activeTaskId)}
    />
  ) : (
    <div className="no-active-task">
      <p>Select a task tab to view details</p>
    </div>
  )}
</div>
</div>
);
};

export default WorkspacePanel;

```

Phase 3: Advanced Features (4-6 weeks)

1. Keyboard Shortcuts

```

// src/hooks/useKeyboardShortcuts.ts
import { useEffect } from 'react';

interface KeyboardShortcutsProps {
  shortcuts: {
    key: string;
    ctrlKey?: boolean;
    altKey?: boolean;
    shiftKey?: boolean;
    action: () => void;
    description: string;
  }[];
  enabled?: boolean;
}

export const useKeyboardShortcuts = ({ shortcuts, enabled = true }:
KeyboardShortcutsProps) => {
  useEffect(() => {
    if (!enabled) return;

    const handleKeyDown = (event: KeyboardEvent) => {
      for (const shortcut of shortcuts) {

```

```

    if (
      event.key.toLowerCase() === shortcut.key.toLowerCase() &&
      !!event.ctrlKey === !!shortcut.ctrlKey &&
      !!event.altKey === !!shortcut.altKey &&
      !!event.shiftKey === !!shortcut.shiftKey
    ) {
      event.preventDefault();
      shortcut.action();
      return;
    }
  }
};

window.addEventListener('keydown', handleKeyDown);

return () => {
  window.removeEventListener('keydown', handleKeyDown);
};
}, [shortcuts, enabled]);

return {
  shortcuts: shortcuts.map(s => ({
    key: s.key,
    ctrlKey: s.ctrlKey,
    altKey: s.altKey,
    shiftKey: s.shiftKey,
    description: s.description
  })))
};
};

```

2. Workspace Layout Persistence

```

// src/components/WorkspaceView.tsx
import React, { useState, useEffect } from 'react';
import { WorkspaceState, Task } from '../types';
import WorkspacePanel from '../WorkspacePanel';
import { useKeyboardShortcuts } from '../hooks/useKeyboardShortcuts';
import { v4 as uuidv4 } from 'uuid';

interface WorkspaceViewProps {
  workspace: WorkspaceState;
  onWorkspaceChange: (workspace: WorkspaceState) => void;
  onNavigateToSearch: () => void;
  fetchTask: (id: string) => Promise<Task>;
}

const WorkspaceView: React.FC<WorkspaceViewProps> = ({
  workspace,
  onWorkspaceChange,
  onNavigateToSearch,
  fetchTask
}) => {
  const [layouts, setLayouts] = useState<string[]>(['default']);

```

```
const [currentLayout, setCurrentLayout] = useState<string>('default');
```

```
// Set up keyboard shortcuts
```

```
useKeyboardShortcuts({
  shortcuts: [
    {
      key: 's',
      ctrlKey: true,
      action: onNavigateToSearch,
      description: 'Go to search'
    },
    {
      key: 'n',
      ctrlKey: true,
      action: () => addPanel(),
      description: 'Add new panel'
    },
    {
      key: 'w',
      ctrlKey: true,
      action: () => {
        if (workspace.activeTaskId) {
          closeTask(workspace.activeTaskId);
        }
      },
      description: 'Close active task'
    }
  ]
});
```

```
// Add a new panel
```

```
const addPanel = () => {
  const updatedWorkspace = {
    ...workspace,
    paneGroups: [
      ...workspace.paneGroups,
      { id: uuidv4(), taskIds: [], activeTaskId: undefined }
    ]
  };
};
```

```
onWorkspaceChange(updatedWorkspace);
};
```

```
// Remove a panel
```

```
const removePanel = (panelId: string) => {
  // Don't remove if it's the last panel
  if (workspace.paneGroups.length <= 1) {
    return;
  }

  const updatedWorkspace = {
    ...workspace,
    paneGroups: workspace.paneGroups.filter(pg => pg.id !== panelId)
  };

  onWorkspaceChange(updatedWorkspace);
};
```

```

// Activate a task in a panel
const activateTask = (panelId: string, taskId: string) => {
  const updatedWorkspace = {
    ...workspace,
    paneGroups: workspace.paneGroups.map(pg =>
      pg.id === panelId ? { ...pg, activeTaskId: taskId } : pg
    ),
    activeTaskId: taskId
  };

  onWorkspaceChange(updatedWorkspace);
};

// Close a task
const closeTask = (taskId: string) => {
  // Find which panel contains this task
  const panelWithTask = workspace.paneGroups.find(pg =>
    pg.taskIds.includes(taskId)
  );

  if (!panelWithTask) return;

  // Remove task from panel
  const updatedTaskIds = panelWithTask.taskIds.filter(id => id !==
taskId);

  // Determine new active task if needed
  let newActiveTaskId = panelWithTask.activeTaskId;
  if (panelWithTask.activeTaskId === taskId) {
    newActiveTaskId = updatedTaskIds.length > 0 ? updatedTaskIds[0] :
undefined;
  }

  // Update workspace
  const updatedWorkspace = {
    ...workspace,
    openTasks: workspace.openTasks.filter(id => id !== taskId),
    paneGroups: workspace.paneGroups.map(pg =>
      pg.id === panelWithTask.id
        ? { ...pg, taskIds: updatedTaskIds, activeTaskId: newActiveTaskId
}
        : pg
    ),
    activeTaskId: workspace.activeTaskId === taskId
      ? newActiveTaskId || workspace.paneGroups[0]?.activeTaskId
      : workspace.activeTaskId
  };

  onWorkspaceChange(updatedWorkspace);
};

// Handle dropping a task into a panel
const handleDropTask = (panelId: string, taskId: string) => {
  // Find which panel currently contains this task
  const sourcePanel = workspace.paneGroups.find(pg =>
    pg.taskIds.includes(taskId)

```

```

);

// If task is already in a panel, remove it from there
let updatedPaneGroups = [...workspace.paneGroups];
if (sourcePanel) {
  updatedPaneGroups = updatedPaneGroups.map(pg =>
    pg.id === sourcePanel.id
      ? {
          ...pg,
          taskIds: pg.taskIds.filter(id => id !== taskId),
          activeTaskId: pg.activeTaskId === taskId
            ? (pg.taskIds.length > 1 ? pg.taskIds.find(id => id !==
taskId) : undefined)
            : pg.activeTaskId
        }
      : pg
  );
}

// Add task to target panel
updatedPaneGroups = updatedPaneGroups.map(pg =>
  pg.id === panelId
    ? {
        ...pg,
        taskIds: [...pg.taskIds, taskId],
        activeTaskId: taskId
      }
    : pg
);

// Update workspace
const updatedWorkspace = {
  ...workspace,
  paneGroups: updatedPaneGroups,
  activeTaskId: taskId
};

onWorkspaceChange(updatedWorkspace);
};

// Save current layout
const saveLayout = (name: string) => {
  // In a real app, you'd save this to backend
  setLayouts([...layouts.filter(l => l !== name), name]);
  setCurrentLayout(name);
  localStorage.setItem(`workspace-layout-${name}`,
JSON.stringify(workspace));
};

// Load a saved layout
const loadLayout = (name: string) => {
  const savedLayout = localStorage.getItem(`workspace-layout-${name}`);
  if (savedLayout) {
    const parsedLayout = JSON.parse(savedLayout);
    onWorkspaceChange(parsedLayout);
    setCurrentLayout(name);
  }
}

```



```

};

return (
  <div className="workspace-view">
    <div className="workspace-toolbar">
      <button onClick={addPanel}>Add Panel</button>

      <div className="layout-controls">
        <select
          value={currentLayout}
          onChange={(e) => loadLayout(e.target.value)}
        >
          {layouts.map(layout => (
            <option key={layout} value={layout}>{layout}</option>
          ))}
        </select>

        <button onClick={() => {
          const name = prompt('Enter layout name', currentLayout);
          if (name) saveLayout(name);
        }}>
          Save Layout
        </button>
      </div>

      <button onClick={onNavigateToSearch}>Search</button>
    </div>

    <div className="workspace-panels">
      {workspace.paneGroups.map(panel => (
        <div key={panel.id} className="panel-container">
          <WorkspacePanel
            id={panel.id}
            taskIds={panel.taskIds}
            activeTaskId={panel.activeTaskId}
            onActivateTask={(taskId) => activateTask(panel.id, taskId)}
            onCloseTask={closeTask}
            onDropTask={(taskId) => handleDropTask(panel.id, taskId)}
            fetchTask={fetchTask}
          />

          {workspace.paneGroups.length > 1 && (
            <button
              className="remove-panel-button"
              onClick={() => removePanel(panel.id)}
            >
              Remove Panel
            </button>
          )}
        </div>
      ))}
    </div>
  </div>
);
};

export default WorkspaceView;

```

3. Vector Search Integration

```
// src/api/vectorSearchService.ts
import apiClient from './client';
import { Task, SearchParams } from '../types';
import { mapApiTaskToTask } from './taskService';

export interface VectorSearchParams {
  query: string;
  filter_list_name?: string;
  filter_folder_name?: string;
  filter_status?: string;
  filter_completed?: boolean;
  hybrid_search: boolean;
  similarity_threshold: number;
  page: number;
  page_size: number;
}

export const performVectorSearch = async (searchParams: SearchParams):
Promise<Task[]> => {
  try {
    const vectorParams: VectorSearchParams = {
      query: searchParams.query || '',
      filter_list_name: searchParams.filters.list,
      filter_folder_name: searchParams.filters.folder,
      filter_status: searchParams.filters.status?.length === 1 ?
searchParams.filters.status[0] : undefined,
      filter_completed: searchParams.filters.status?.includes('completed')
|| undefined,
      hybrid_search: true, // Enable hybrid search by default
      similarity_threshold: 0.7,
      page: 1,
      page_size: 50
    };

    const response = await apiClient.post('/todos/vector-search',
vectorParams);
    return response.data.map(mapApiTaskToTask);
  } catch (error) {
    console.error('Vector search failed:', error);
    throw error;
  }
};

// Enhanced SearchView component with vector search
export const useVectorSearch = (searchParams: SearchParams) => {
  const [results, setResults] = useState<Task[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const search = async () => {
```

```

    if (!searchParams.query) {
      setResults([]);
      return;
    }

    setLoading(true);
    setError(null);

    try {
      const searchResults = await performVectorSearch(searchParams);
      setResults(searchResults);
    } catch (err) {
      setError('Vector search failed. Falling back to regular search.');
```

// Could implement fallback to regular search here

```

    } finally {
      setLoading(false);
    }
  };

  search();
}, [searchParams]);

return { results, loading, error };
};

```

Phase 4: Performance Optimizations (2-4 weeks)

1. Virtualized Lists for Search Results

```

// src/components/VirtualizedTaskList.tsx
import React, { useCallback } from 'react';
import { FixedSizeList as List } from 'react-window';
import AutoSizer from 'react-virtualized-auto-sizer';
import { Task } from '../types';

interface VirtualizedTaskListProps {
  tasks: Task[];
  onTaskClick: (task: Task) => void;
  openTaskIds: string[];
}

const VirtualizedTaskList: React.FC<VirtualizedTaskListProps> = ({
  tasks,
  onTaskClick,
  openTaskIds
}) => {
  const Row = useCallback(({ index, style }: { index: number; style:
React.CSSProperties }) => {
    const task = tasks[index];

```

```

    return (
      <div
        style={style}
        className={`task-item ${openTaskIds.includes(task.id) ? 'open' :
''}`}
        onClick={() => onTaskClick(task)}
      >
        <div className="task-title">
          {task.title}
          {openTaskIds.includes(task.id) && (
            <span className="open-indicator">Open</span>
          )}
        </div>

        <div className="task-metadata">
          <span className={`status ${task.status}`}>
            {task.status}
          </span>
          <span className="list">
            {task.list}
          </span>
          {task.dueDate && (
            <span className="due-date">
              Due: {new Date(task.dueDate).toLocaleDateString()}
            </span>
          )}
        </div>
      </div>
    );
  }, [tasks, onTaskClick, openTaskIds]);

  return (
    <div className="virtualized-task-list" style={{ height: '100%', width:
'100%' }}>
      <AutoSizer>
        ({({ height, width }) => (
          <List
            height={height}
            width={width}
            itemCount={tasks.length}
            itemSize={80} // Adjust based on your row height
          >
            {Row}
          </List>
        )}
      </AutoSizer>
    </div>
  );
};

export default VirtualizedTaskList;

```

2. Debounced Search

```
// src/hooks/useDebounce.ts
import { useState, useEffect } from 'react';

export function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

// Usage in SearchBar component
const SearchBar: React.FC<SearchBarProps> = ({ searchParams, onSearchChange
}) => {
  const [inputValue, setInputValue] = useState(searchParams.query);
  const debouncedValue = useDebounce(inputValue, 300);

  useEffect(() => {
    if (debouncedValue !== searchParams.query) {
      onSearchChange(debouncedValue);
    }
  }, [debouncedValue, onSearchChange, searchParams.query]);

  return (
    <input
      type="text"
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      placeholder="Search tasks..."
    />
  );
};
```

3. Memoized Components

```
// Example of memoizing a component
import React, { memo, useMemo } from 'react';

const TaskItem = memo(({ task, onClick, isOpen }: TaskItemProps) => {
  // Component implementation
  return (
    <div
      className={`task-item ${isOpen ? 'open' : ''}`}
      onClick={() => onClick(task)}
    >
```

```

    >
    { /* Task content */ }
  </div>
);
});

// Using useMemo for expensive calculations
const SearchResults: React.FC<SearchResultsProps> = ({ tasks, searchParams
}) => {
  const filteredTasks = useMemo(() => {
    return tasks.filter(task => {
      // Apply filters
      return true; // Replace with actual filtering logic
    });
  }, [tasks, searchParams.filters]);

  const sortedTasks = useMemo(() => {
    return [...filteredTasks].sort((a, b) => {
      // Apply sorting
      return 0; // Replace with actual sorting logic
    });
  }, [filteredTasks, searchParams.sortBy, searchParams.sortDirection]);

  return (
    <div className="search-results">
      {sortedTasks.map(task => (
        <TaskItem
          key={task.id}
          task={task}
          onClick={handleTaskClick}
          isOpen={openTaskIds.includes(task.id)}
        />
      ))}
    </div>
  );
};

```

Project Structure

```

src/
├── api/
│   ├── client.ts           # API client setup
│   ├── taskService.ts      # Task API methods
│   ├── workspaceService.ts # Workspace API methods
│   └── vectorSearchService.ts # Vector search API methods
├── components/
│   ├── App.tsx             # Main application component
│   ├── SearchView.tsx      # Search interface
│   ├── SearchBar.tsx       # Search input
│   ├── FilterPanel.tsx     # Search filters
│   ├── SearchResults.tsx   # Search results display
│   └── TaskItem.tsx        # Individual task in results

```

	WorkspaceView.tsx	# Workspace interface
	WorkspacePanel.tsx	# Panel component
	TaskTab.tsx	# Tab for open tasks
	TaskDetail.tsx	# Task detail view
	VirtualizedTaskList.tsx	# Performance-optimized list
	hooks/	
	useDebounce.ts	# Debounced input hook
	useKeyboardShortcuts.ts	# Keyboard shortcuts hook
	useVectorSearch.ts	# Vector search hook
	types/	
	index.ts	# TypeScript type definitions
	utils/	
	dateUtils.ts	# Date formatting utilities
	searchUtils.ts	# Search helper functions
	styles/	
	index.css	# Global styles
	search.css	# Search view styles
	workspace.css	# Workspace view styles
	index.tsx	# Application entry point

Implementation Timeline

Phase 1: API Integration (4-6 weeks)

- Week 1-2: Set up API client and basic task service
- Week 3-4: Implement search functionality with API
- Week 5-6: Add workspace persistence with API

Phase 2: Enhanced UI Components (4-6 weeks)

- Week 1-2: Improve task detail view with markdown support
- Week 3-4: Enhance search results with grouping and sorting
- Week 5-6: Implement advanced workspace panel features

Phase 3: Advanced Features (4-6 weeks)

- Week 1-2: Add keyboard shortcuts
- Week 3-4: Implement workspace layout persistence
- Week 5-6: Integrate vector search capabilities

Phase 4: Performance Optimizations (2-4 weeks)

- Week 1-2: Add virtualized lists for search results
- Week 3-4: Implement debouncing and memoization

Dependencies

```
{
  "dependencies": {
    "@types/node": "^16.18.0",
    "@types/react": "^18.0.0",
    "@types/react-dom": "^18.0.0",
    "axios": "^1.3.4",
    "date-fns": "^2.29.3",
    "react": "^18.2.0",
    "react-dnd": "^16.0.1",
    "react-dnd-html5-backend": "^16.0.1",
    "react-dom": "^18.2.0",
    "react-markdown": "^8.0.5",
    "react-scripts": "5.0.1",
    "react-virtualized-auto-sizer": "^1.0.7",
    "react-window": "^1.8.8",
    "typescript": "^4.9.5",
    "uuid": "^9.0.0"
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "@types/jest": "^27.5.2",
    "@types/react-window": "^1.8.5",
    "@types/uuid": "^9.0.1"
  }
}
```

Future Enhancements

1. Advanced Search Features

- Natural language query parsing
- Saved searches and search history

- Search suggestions based on past queries
- Fuzzy matching for typo tolerance

2. Task Management

- Task creation and editing
- Task relationships (parent/child, dependencies)
- Batch operations on multiple tasks
- Task templates for common patterns

3. Visualization and Analytics

- Task distribution charts by folder, list, status
- Due date calendar view
- Productivity trends over time
- Tag cloud visualization

4. Collaboration Features

- Shared workspaces
- Task commenting
- Activity feed
- User permissions and roles

5. Integration with External Tools

- Calendar integration
- Email integration
- Mobile app synchronization
- Browser extension for quick capture

Implementation Notes

API Integration Strategy

The implementation will follow these principles:

1. **Separation of concerns:** API logic is isolated in service files
2. **Type safety:** Strong TypeScript typing for API requests and responses
3. **Error handling:** Consistent error handling with fallbacks
4. **Caching:** Appropriate caching of API responses for performance
5. **Mapping:** Clear mapping between API and application data models

UI/UX Considerations

1. **Progressive loading:** Show content as soon as possible with loading indicators
2. **Error states:** Clear error messages with recovery options
3. **Empty states:** Helpful guidance when no data is available
4. **Accessibility:** Proper ARIA attributes and keyboard navigation
5. **Responsive design:** Adaptable layout for different screen sizes

Performance Optimization Techniques

1. **Code splitting:** Load components only when needed
2. **Virtualization:** Render only visible items in long lists
3. **Memoization:** Avoid unnecessary re-renders and calculations
4. **Debouncing:** Prevent excessive API calls during user input
5. **Lazy loading:** Defer loading of non-critical resources